

Análisis

1. Tabla de propiedades

Clase	Modificador	Atributo	Tipo de Dato	Propósito
Comprador	private	nombre	String	Almacenar el nombre del comprador activo.
Comprador	private	email	String	Guardar la dirección de correo electrónico del comprador.
Comprador	private	cantidadBoletos	int	Registrar el número de boletos que el comprador desea comprar.
Comprador	private	presupuesto	double	Almacenar la cantidad máxima de dinero disponible por el comprador.
Comprador	private	ticketAsignado	Ticket	Guardar la referencia al objeto ticket generado para la compra.
Ticket	private	numero	int	Almacenar el número aleatorio asignado (1 a 15,000).
Localidad	private	idLocalidad	int	Identificar la localidad (1, 5, 10).
Localidad	private	nombre	String	Guardar la descripción de la localidad.
Localidad	private	precio	double	Almacenar el precio unitario de esa localidad.
Localidad	private	capacidadMaxima	int	Definir el límite de 20 espacios de capacidad por localidad.
Localidad	private	boletosVendidos	int	Llevar el conteo acumulado de boletos vendidos por localidad.
VentasControlador	private	compradorActivo	Comprador	Referencia al comprador activo registrado en ese momento.
VentasControlador	private	localidades	Localidad[]	Guardar el conjunto de las 3 localidades disponibles.
VentasControlador	private	random	Random	Generar números aleatorios para asignar tickets y seleccionar localidades.

VentasVista	private	scanner	Scanner	Capturar los datos de entrada desde la consola.
-------------	---------	---------	---------	---

2. Tabla de métodos

Clase	Visibilidad	Tipo Retorno	Método y Parámetros	Función
Comprador	public	—	Comprador(String nombre, String email, int cantidad, double presupuesto)	Inicializar un nuevo comprador con sus datos básicos.
Comprador	public	String	getNombre()	Retornar el nombre del comprador.
Comprador	public	String	getEmail()	Retornar el correo electrónico del comprador.
Comprador	public	int	getCantidadBoletos()	Retornar la cantidad de boletos solicitada.
Comprador	public	double	getPresupuesto()	Retornar el presupuesto disponible.
Comprador	public	Ticket	getTicketAsignado()	Retornar el objeto ticket asociado al comprador.
Comprador	public	void	setTicketAsignado(Ticket ticket)	Asignar el objeto ticket generado para el comprador.
Ticket	public	—	Ticket(int numero)	Inicializar el ticket con el número asignado.
Ticket	public	int	getNumero()	Retornar el número almacenado en el ticket.
Ticket	public	boolean	esElegibleParaCompra()	Generar los límites a y b y valida si el número está en rango.
Localidad	public	—	Localidad(int idLocalidad, String nombre, double precio, int capacidad)	Inicializar datos y límites de una localidad.
Localidad	public	int	getIdLocalidad()	Retornar el identificador numérico de la localidad.

Localidad	public	String	getNombre()	Retornar el nombre descriptivo de la localidad.
Localidad	public	double	getPrecio()	Retornar el precio unitario del boleto.
Localidad	public	int	getBoletosDisponibles()	Retornar la cantidad de boletos disponibles.
Localidad	public	int	getBoletosVendidos()	Retornar la cantidad de boletos vendidos.
Localidad	public	void	venderBoletos(int cantidad)	Incrementar el contador de boletos vendidos.
Localidad	public	double	getRecaudacion()	Retornar el total de dinero recaudado.
VentasControlador	public	—	VentasControlador()	Inicializar las 3 localidades por defecto.
VentasControlador	public	void	registrarComprador(Comprador nuevoComprador)	Asignar el comprador activo en el sistema.
VentasControlador	public	String	procesarSolicitudBoletos()	Coordinar la compra: asignar ticket, delegar validación al ticket y elegir localidad.
VentasControlador	public	String	obtenerDisponibilidadTotal()	Generar reporte de disponibilidad global de boletos.
VentasControlador	public	String	obtenerDisponibilidadIndividual(int idLocalidad)	Generar reporte de disponibilidad de una localidad específica.
VentasControlador	public	String	obtenerReporteCaja()	Generar el reporte financiero total de ventas.
VentasVista	public	—	VentasVista()	Inicializar la captura de entradas por consola.
VentasVista	public	int	mostrarMenu()	Desplegar el menú y retornar la opción seleccionada.
VentasVista	public	void	mostrarMensaje(String mensaje)	Imprimir mensajes en consola.

VentasVista	public	Comprador	leerComprador()	Solicitar datos del usuario y retornar un objeto Comprador.
VentasVista	public	int	pedirIdLocalidad()	Solicitar y retornar el id de localidad a consultar.

3. Preguntas

6. ¿Cómo clasifica cada una de sus clases según el patrón MVC (Model, View, Controller)? Justifique brevemente el rol que cumple cada una dentro de esa clasificación.

- La clase Comprador es un Modelo ya que representa la entidad del usuario interesado en adquirir boletos. Su función principal es encapsular y almacenar el estado de los datos del comprador activo.
- La clase Localidad es un Modelo ya que almacena y gestiona el estado persistente de cada zona física del estadio.
- La clase Ticket es un Modelo ya que contiene la lógica para validar si su propio numero es válido.
- La clase VentasControlador es un Controlador ya actúa como el intermediario entre la Vista y los Modelos. Contiene la lógica de negocio y las reglas del sistema.
- La clase VentasVista es una Vista ya maneja exclusivamente la interfaz de usuario por consola. Su rol es capturar las entradas de teclado. desplegar el menú principal y mostrar los reportes y mensajes formateados que el controlador le entrega.

7. ¿Qué ventaja ofrece separar la lógica de negocio (Model) de la interacción con el usuario (View/Controller) en este sistema? ¿Qué problema evita esta separación si en el futuro se necesita modificar el algoritmo de asignación de tickets?

La ventaja principal es la separación de responsabilidades porque las clases están relacionadas y si alguna de estas sufriera algún cambio no debería afectar el funcionamiento de las demás. Al crear una clase Ticket (Modelo) se asegura que si se cambia la lógica de generación del numero o de validación, solo se deba de modificar el algoritmo de esa clase sin afectar a los controladores y vistas.

Diseño

